

Notes:

<https://projects.100xdevs.com/tracks/http-deep-dive/HTTP-Deep-dive-2>

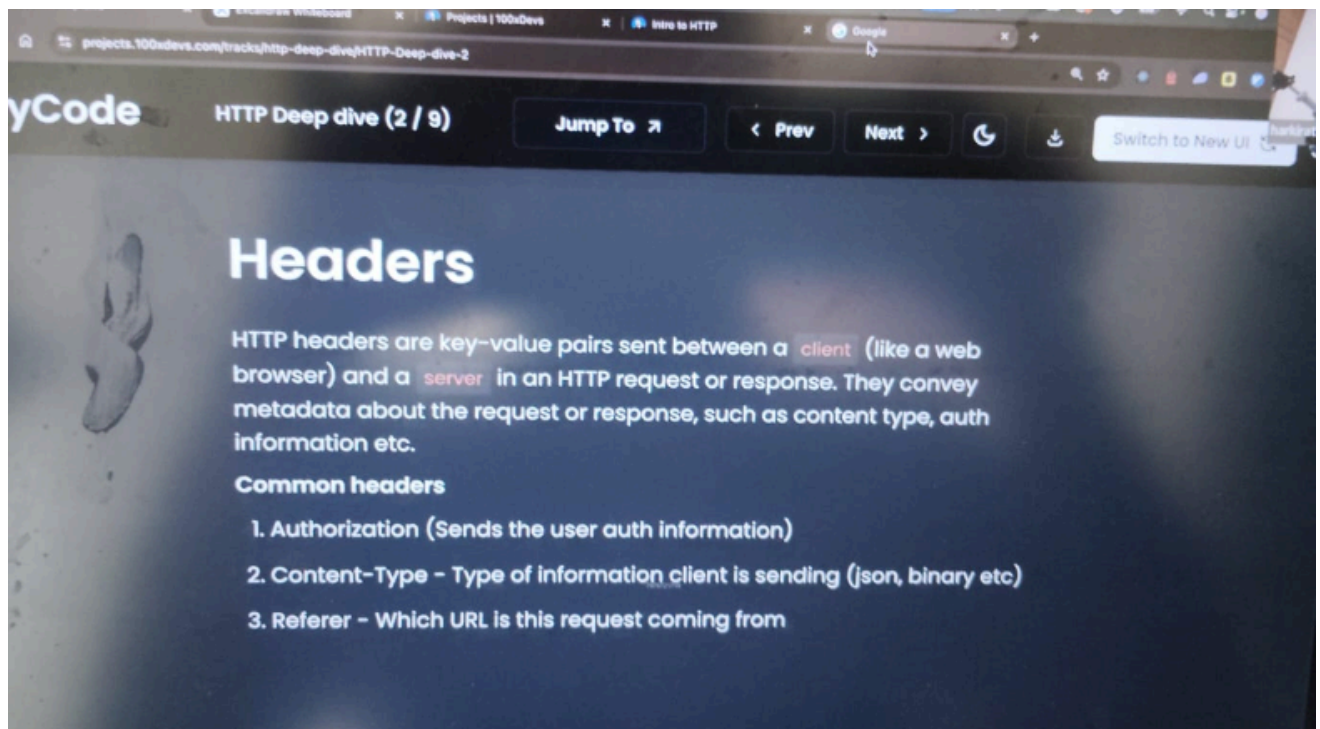
1. Request and Response Headers

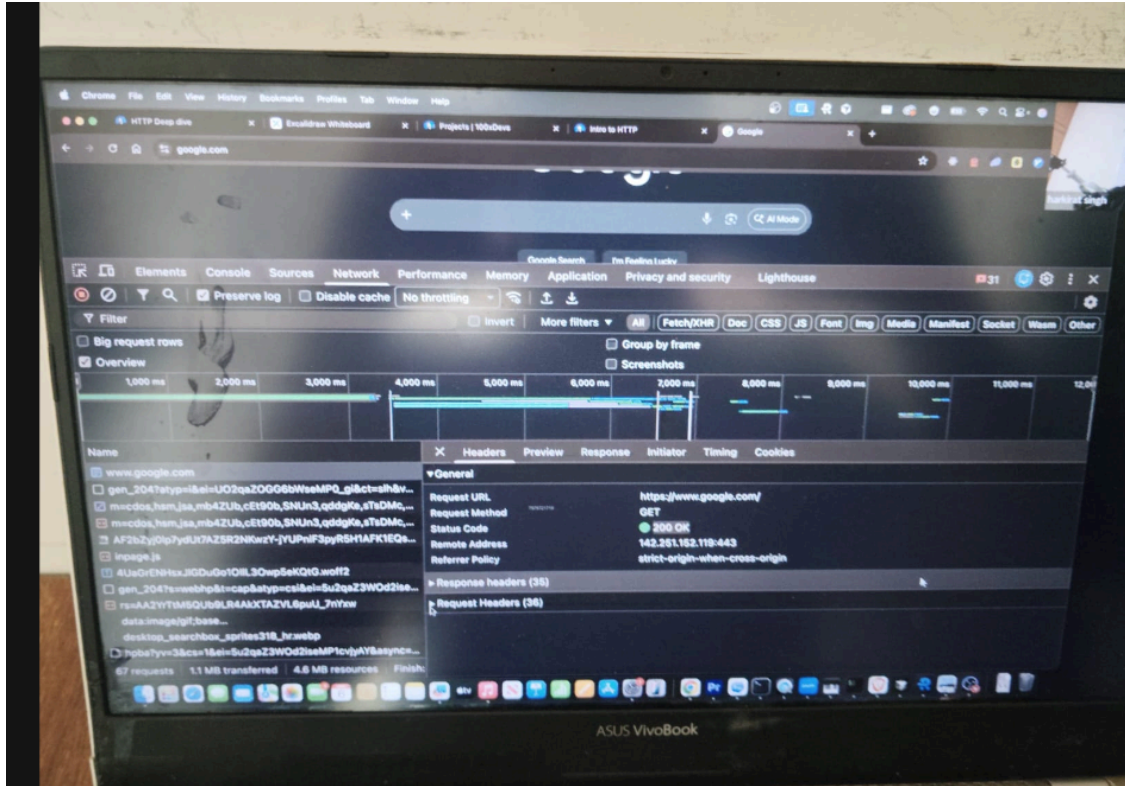
Request Headers: These are the headers that the **client (browser or application)** sends to the **server** along with an HTTP request. They provide additional information about the request, such as the content type, authentication details, and browser information.

Response Headers: These are the headers that the **server sends back to the client** along with the HTTP response. They contain information about the server, the returned data, caching rules, cookies, and other metadata.

Simple way to remember:

- Request Headers → Client → Server
- Response Headers → Server → Client





1. Cookies:

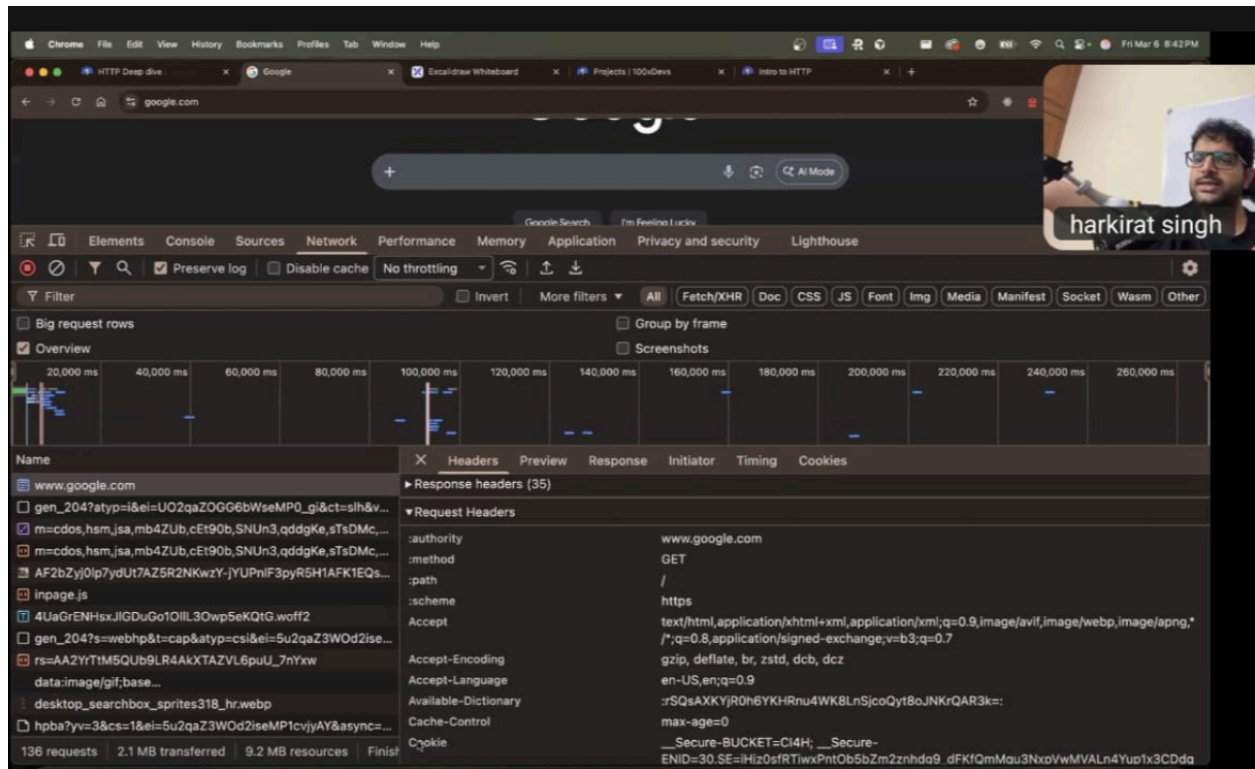
Cookies are small pieces of data stored in the user's browser. They are mainly used for **user authentication and session management**.

When a user logs in to a website, the **server creates a cookie and sends it to the browser**. After that, the **browser automatically includes this cookie in the request headers** whenever it sends a request to the same server.

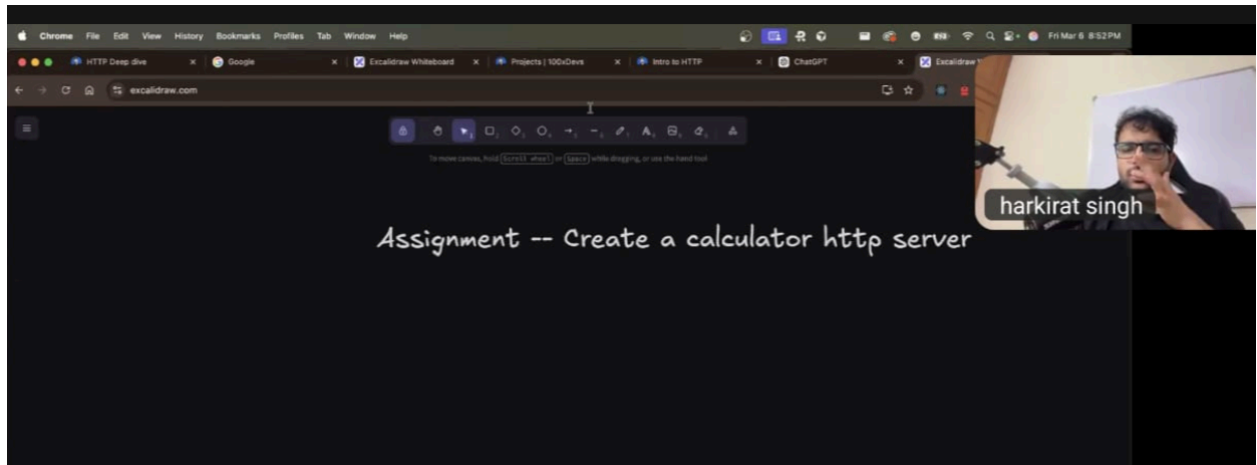
This helps the server **identify and authenticate the user** without requiring them to log in again.

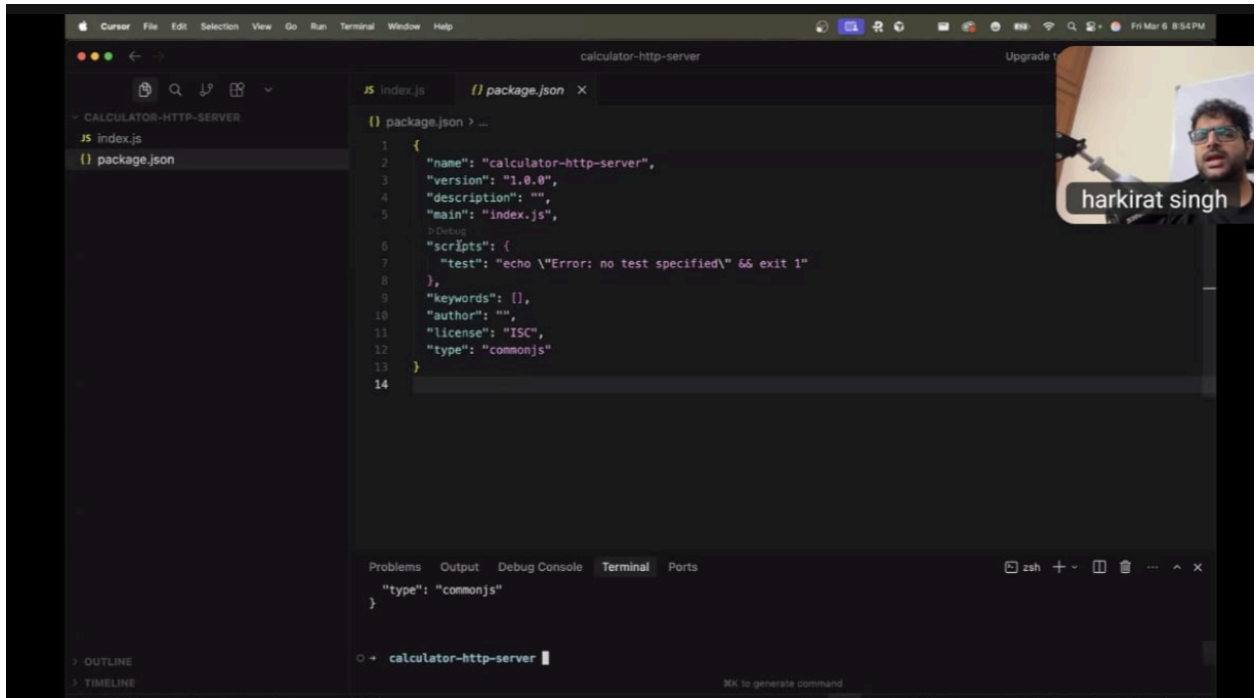
! Important:

If someone gets access to your authentication cookie, they may be able to **access your account on the server**, which is why cookies must be protected using security measures like **HttpOnly, Secure, and SameSite flags**.



2. Create a HTTP Server

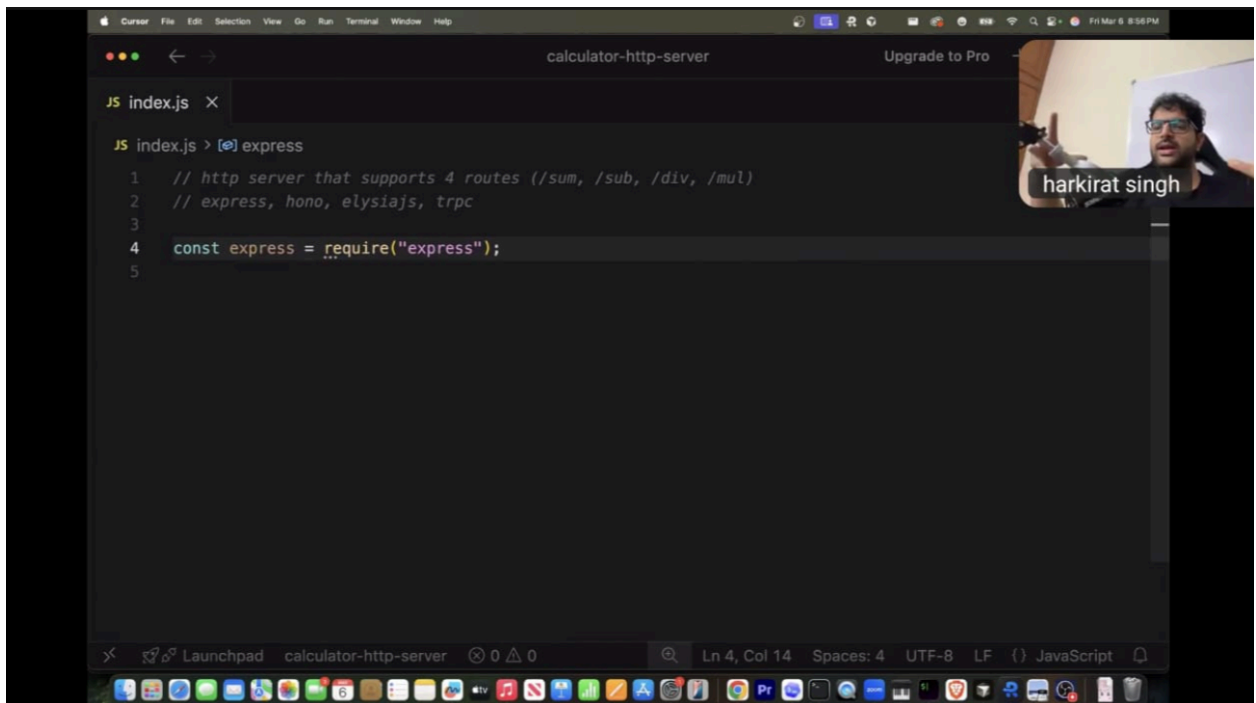




The screenshot shows a code editor with the file `package.json` open. The file contains the following JSON:

```
{
  "name": "calculator-http-server",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \\\"Error: no test specified\\\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

The editor interface includes a sidebar on the left with a file explorer showing `index.js` and `package.json`. The bottom panel shows the `Terminal` tab with a `zsh` prompt. A video feed of a person is visible in the top right corner.

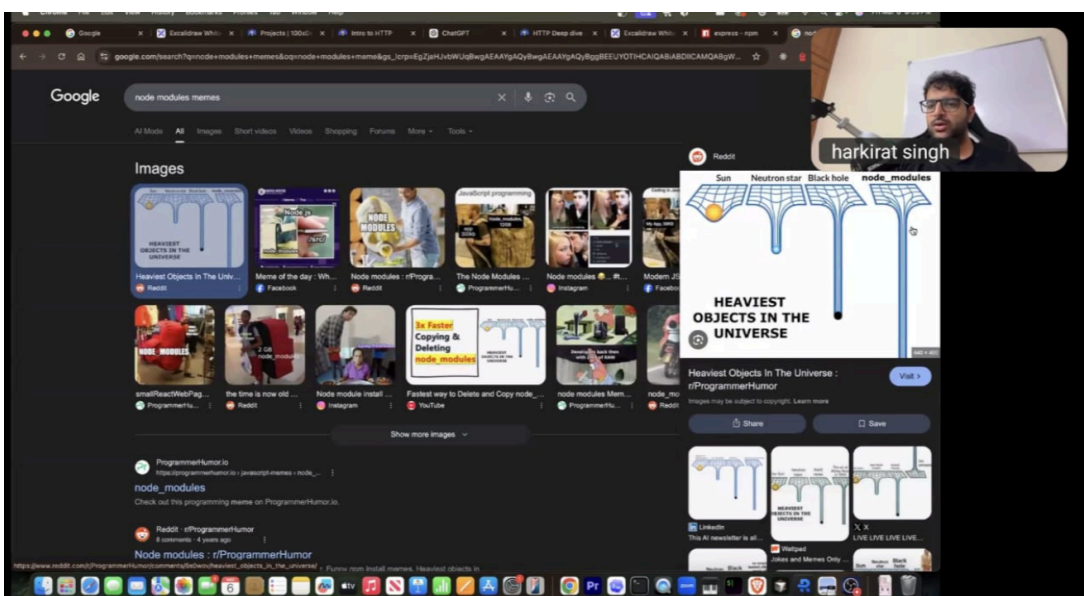
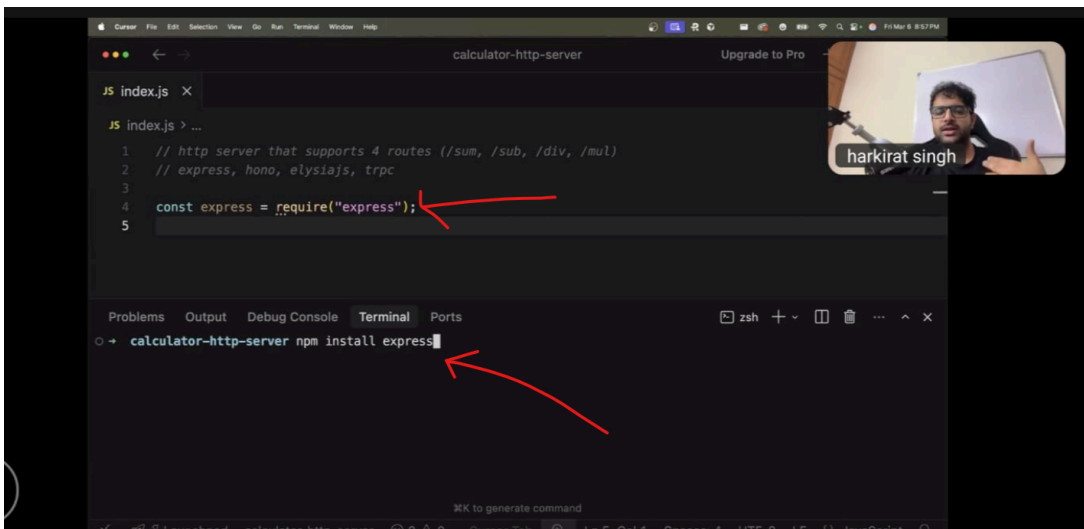
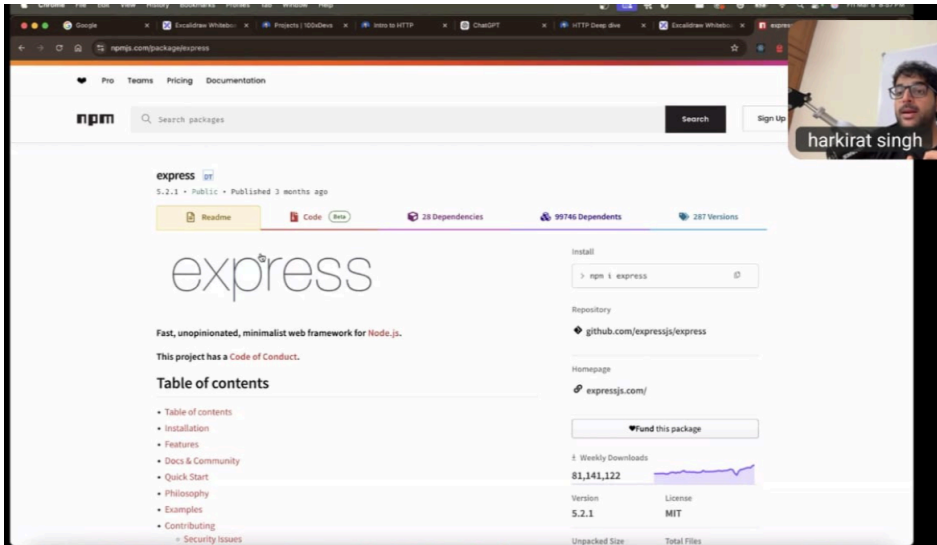


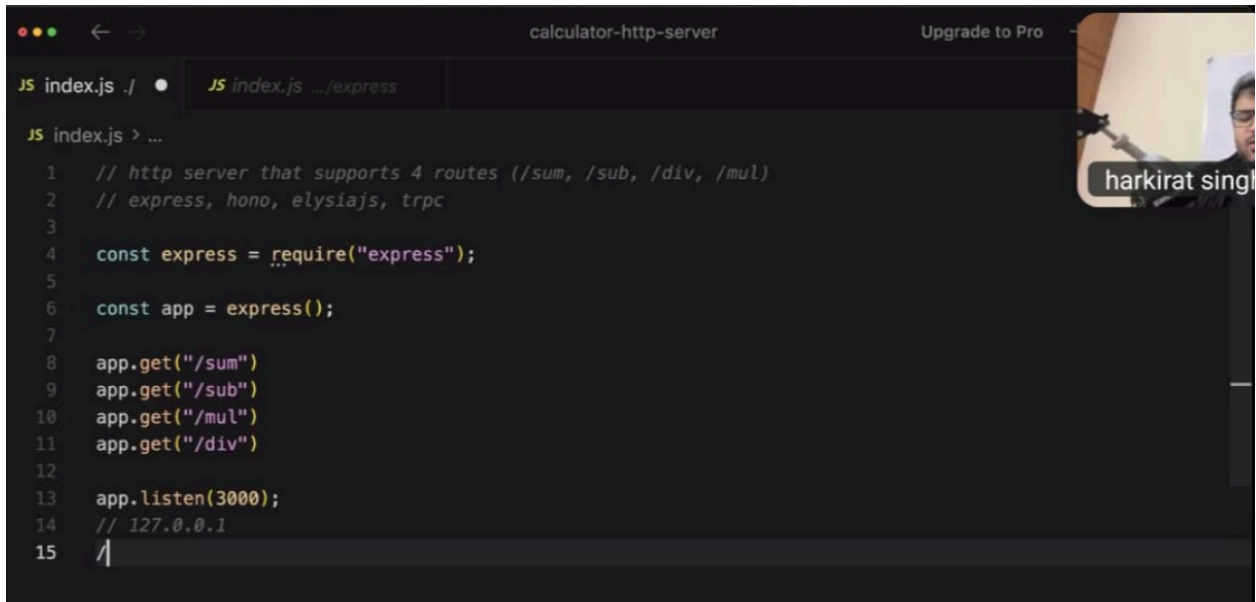
The screenshot shows the same code editor with the file `index.js` open. The file contains the following JavaScript code:

```
// http server that supports 4 routes (/sum, /sub, /div, /mul)
// express, hono, elysiajs, trpc

const express = require("express");
```

The editor interface is similar to the first screenshot, but the sidebar now shows `index.js` as the active file. The bottom panel shows the `Terminal` tab with a `zsh` prompt. A video feed of a person is visible in the top right corner.





```
calculator-http-server Upgrade to Pro
JS index.js ./ JS index.js .../express
JS index.js > ...
1 // http server that supports 4 routes (/sum, /sub, /div, /mul)
2 // express, hono, elysiajs, trpc
3
4 const express = require("express");
5
6 const app = express();
7
8 app.get("/sum")
9 app.get("/sub")
10 app.get("/mul")
11 app.get("/div")
12
13 app.listen(3000);
14 // 127.0.0.1
15 |
```

127.0.0.1 and localhost

- **127.0.0.1** is the **IP address of your local machine** (your own computer). It is also called the **loopback address**.
- <http://localhost:3000> is a **domain name that refers to the same local machine**.

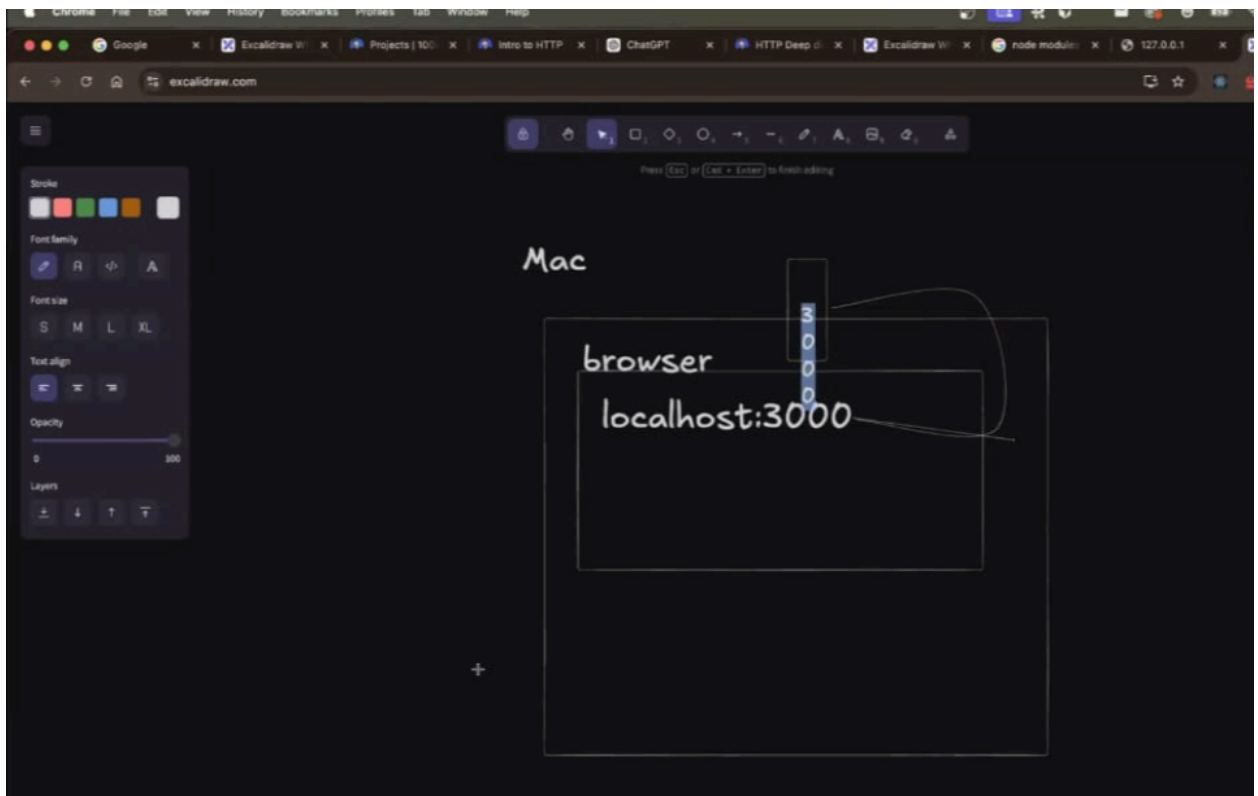
When you open <http://localhost:3000> in your browser, the browser first converts **localhost** into the IP address **127.0.0.1** using DNS or the hosts file.

Then the request is sent to **port 3000 on your own computer**, where your local server (for example a Node.js or Next.js server) is running.

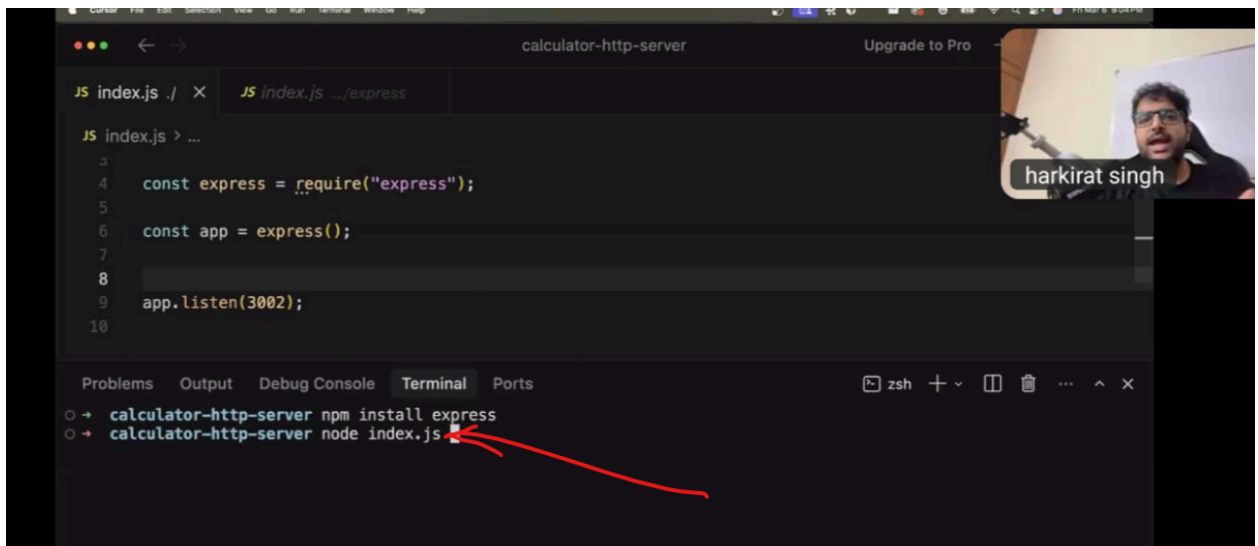
Simple flow:

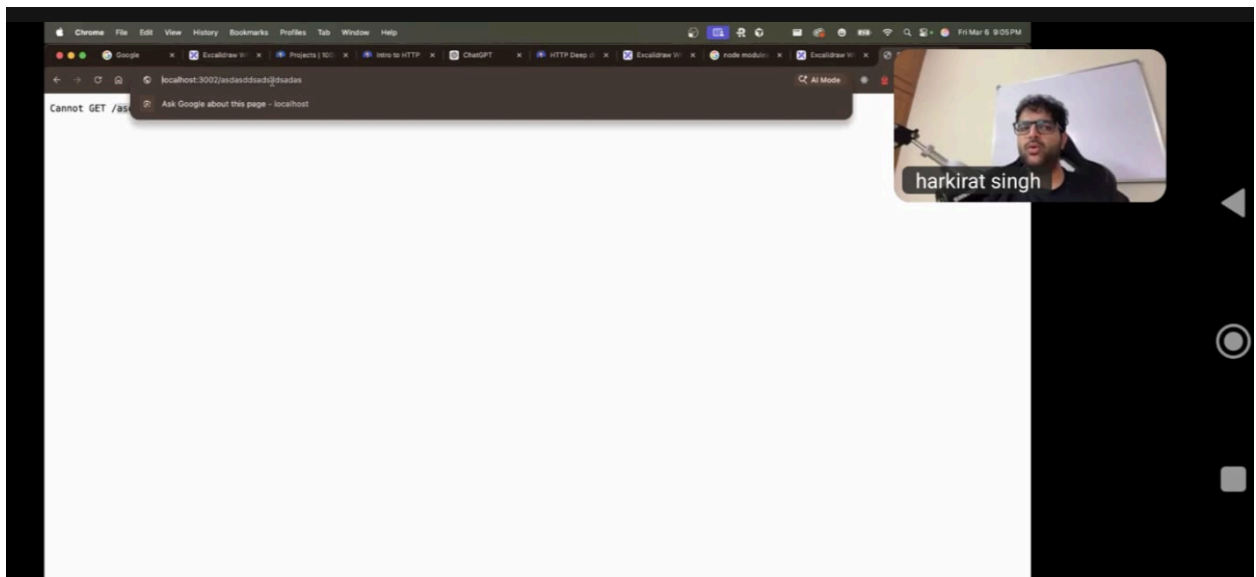
Browser → localhost → resolves to 127.0.0.1 → request goes to your local server (port 3000)

So both **127.0.0.1** and **localhost** point to the **same machine (your computer)**. 🚀

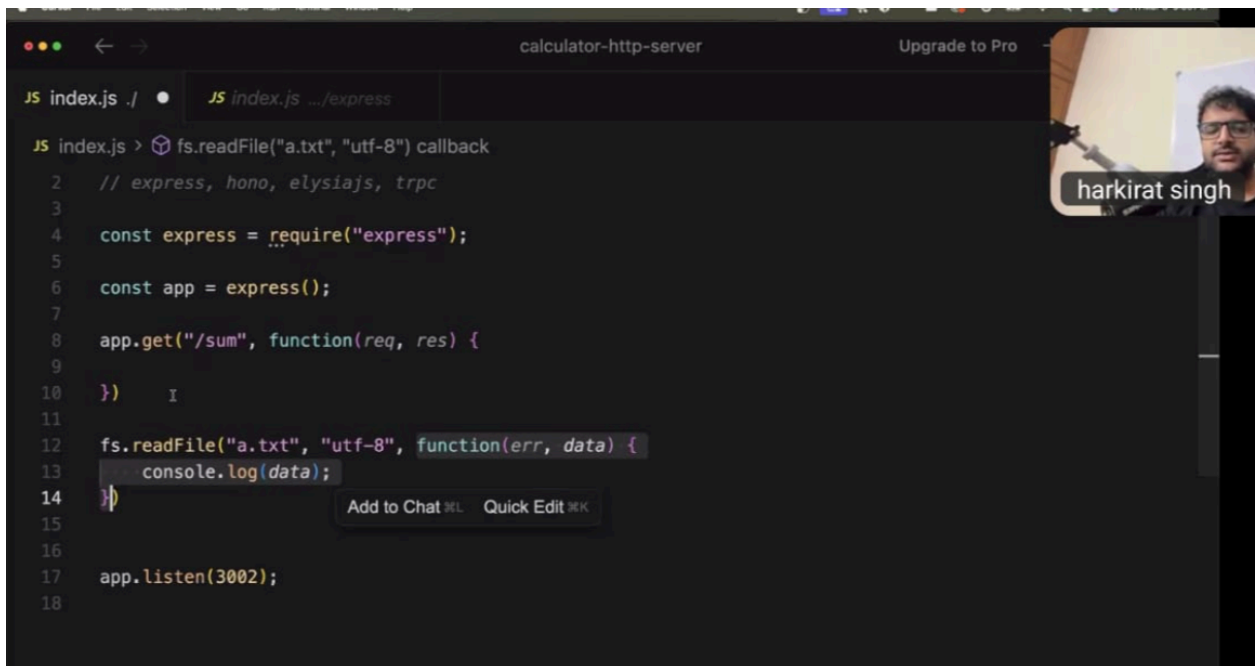


3. To run a server

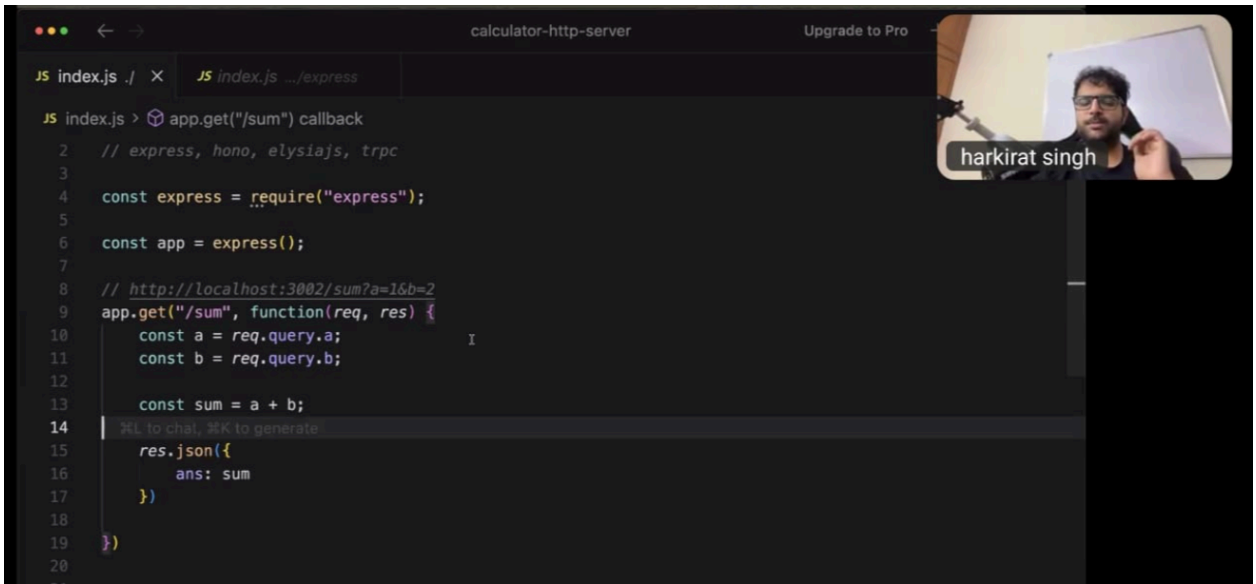




4.



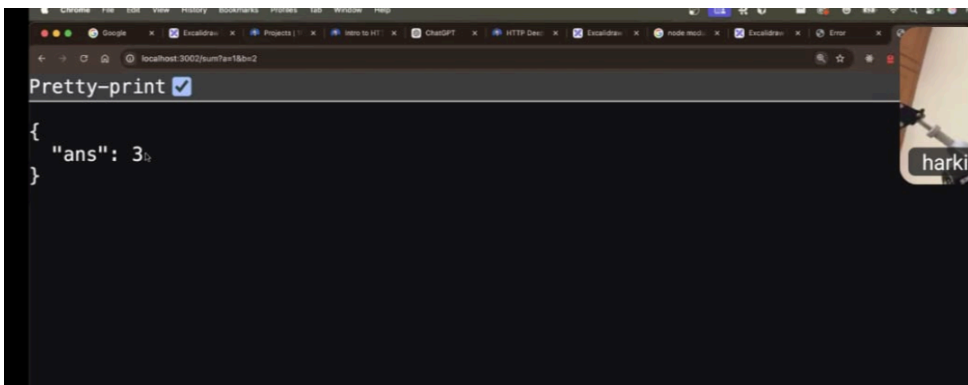
5. Query params



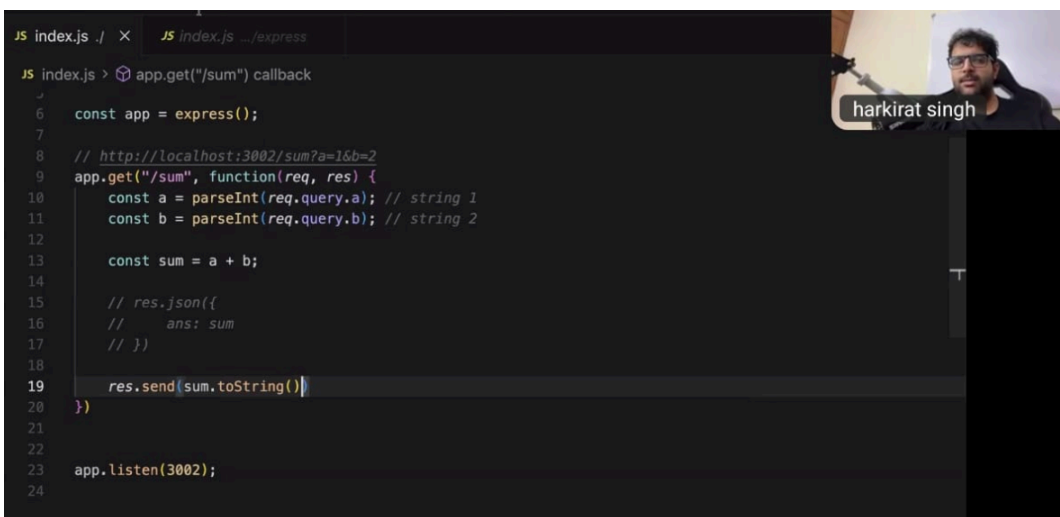
The VS Code editor shows the file `index.js` with the following code:

```
JS index.js > app.get("/sum") callback
2 // express, hono, elysiajs, trpc
3
4 const express = require("express");
5
6 const app = express();
7
8 // http://localhost:3002/sum?a=1&b=2
9 app.get("/sum", function(req, res) {
10   const a = req.query.a;
11   const b = req.query.b;
12
13   const sum = a + b;
14   res.json({
15     ans: sum
16   })
17 }
18
19
20
```

A video inset in the top right corner shows a man with glasses and a beard, identified as harkirat singh, pointing at a whiteboard.



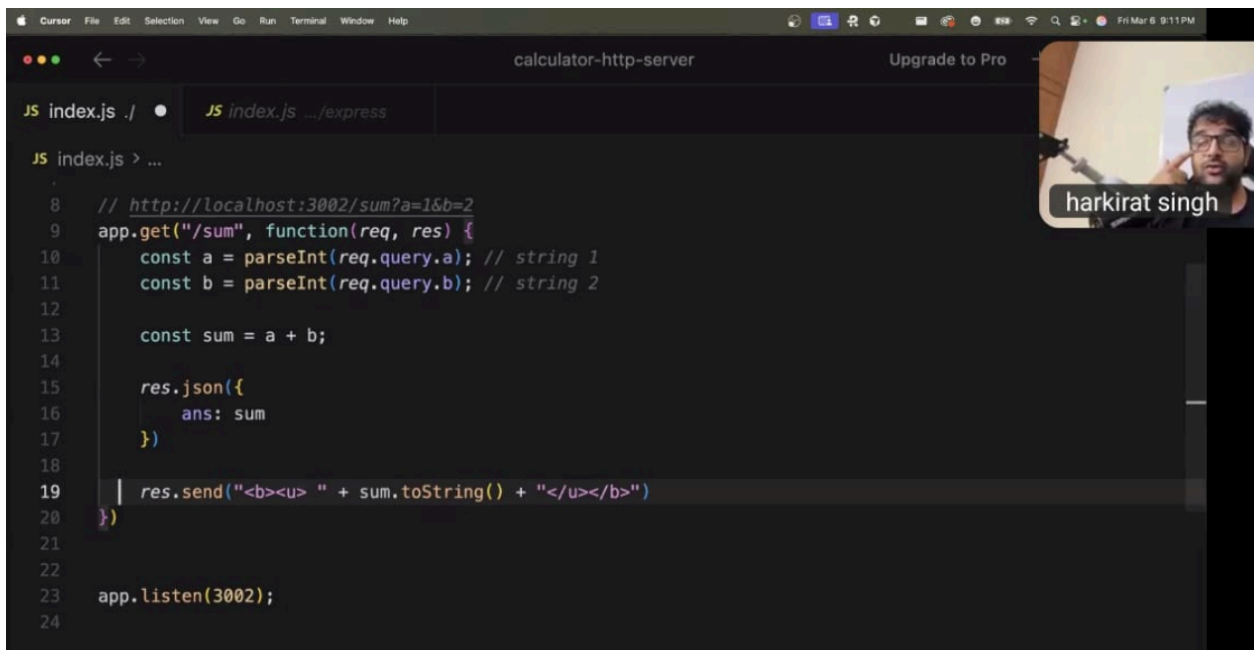
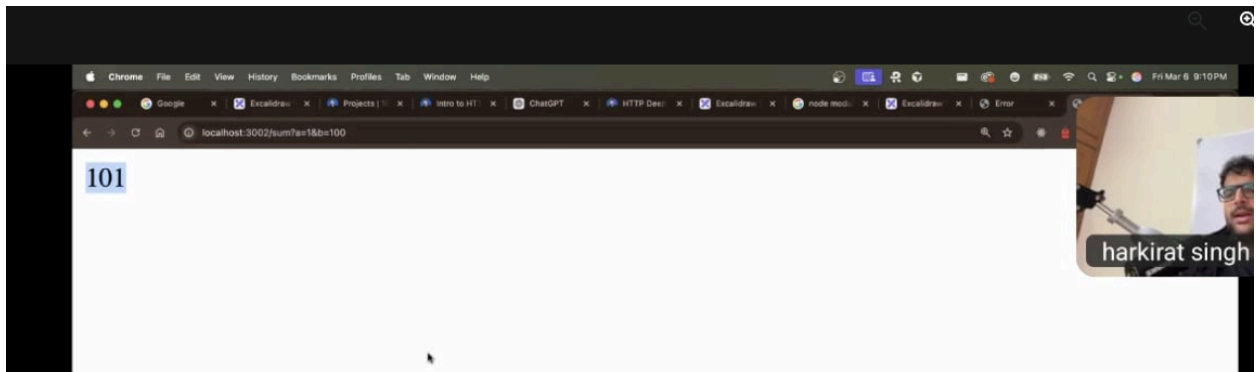
The Chrome DevTools console shows the response of the `localhost:3002/sum?a=1&b=2` request. The response is a JSON object: `{ "ans": 3 }`. The "Pretty-print" button is checked.



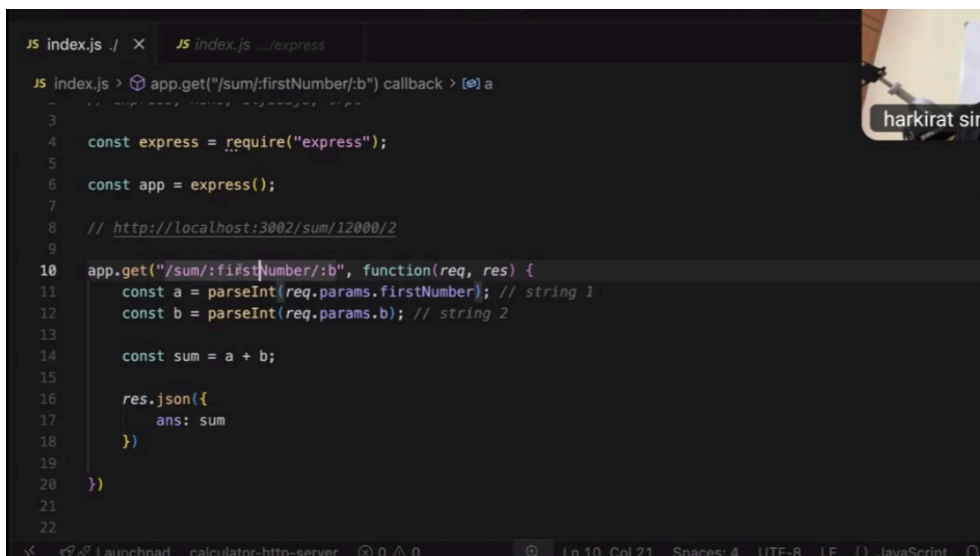
The VS Code editor shows the file `index.js` with the following code:

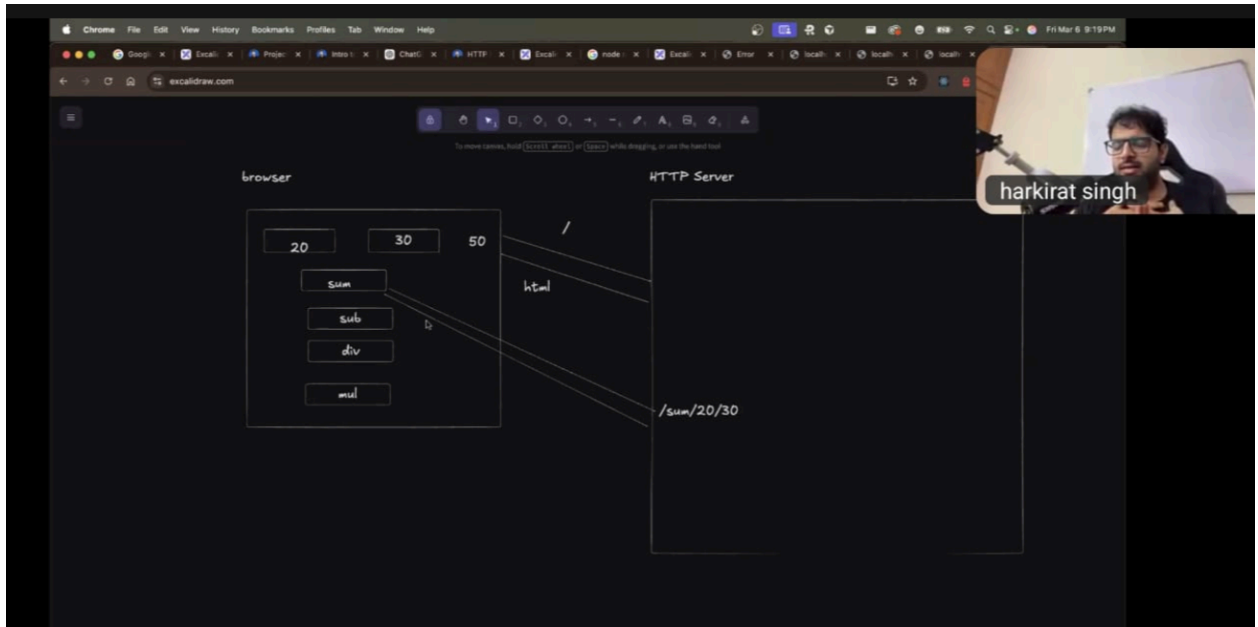
```
JS index.js > app.get("/sum") callback
6 const app = express();
7
8 // http://localhost:3002/sum?a=1&b=2
9 app.get("/sum", function(req, res) {
10   const a = parseInt(req.query.a); // string 1
11   const b = parseInt(req.query.b); // string 2
12
13   const sum = a + b;
14
15   // res.json({
16   //   ans: sum
17   // })
18   res.send(sum.toString())
19 }
20
21
22
23 app.listen(3002);
24
```

A video inset in the top right corner shows a man with glasses and a beard, identified as harkirat singh, pointing at a whiteboard.



6. Path Parameter





7. Fetch API

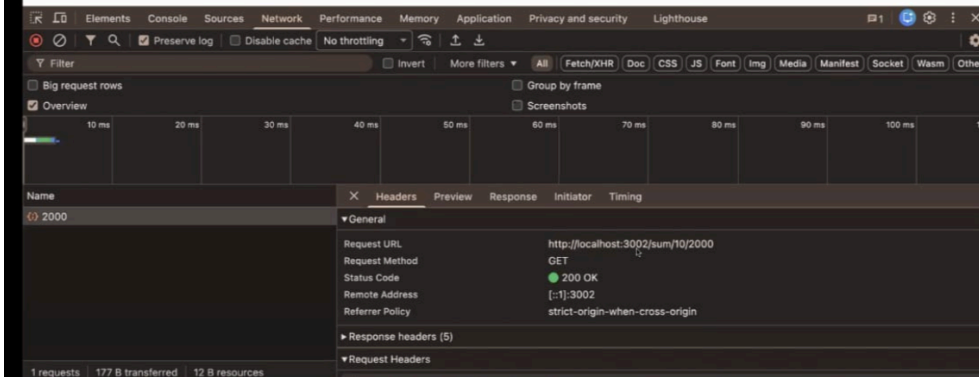
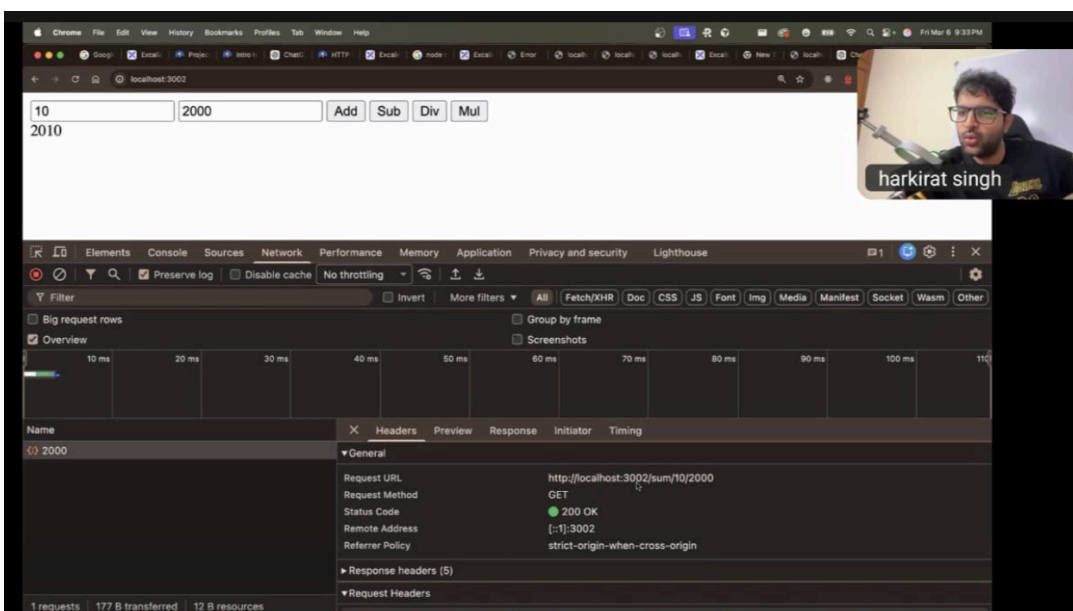
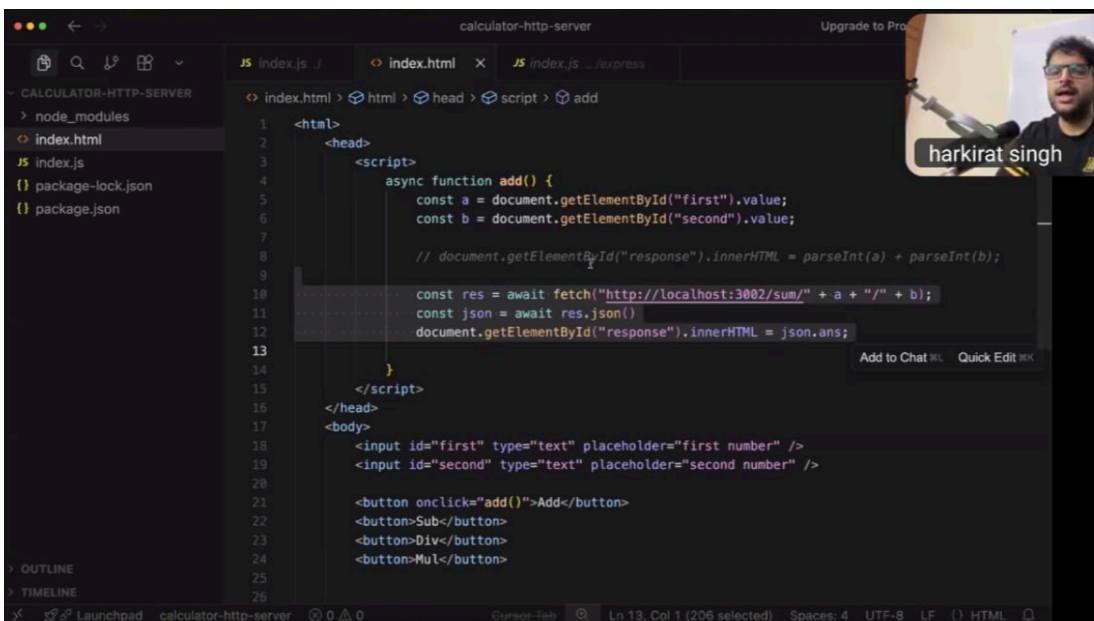
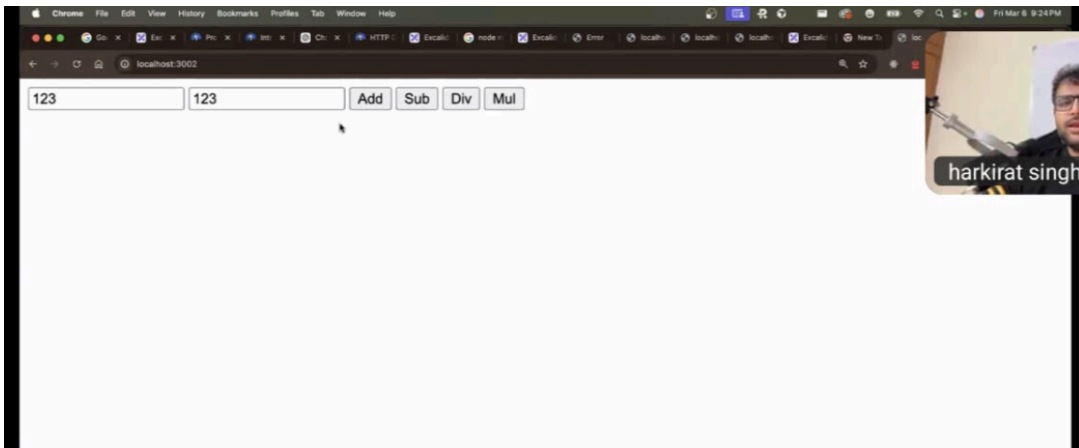
```
calculator-http-server
Upgrade to Pro

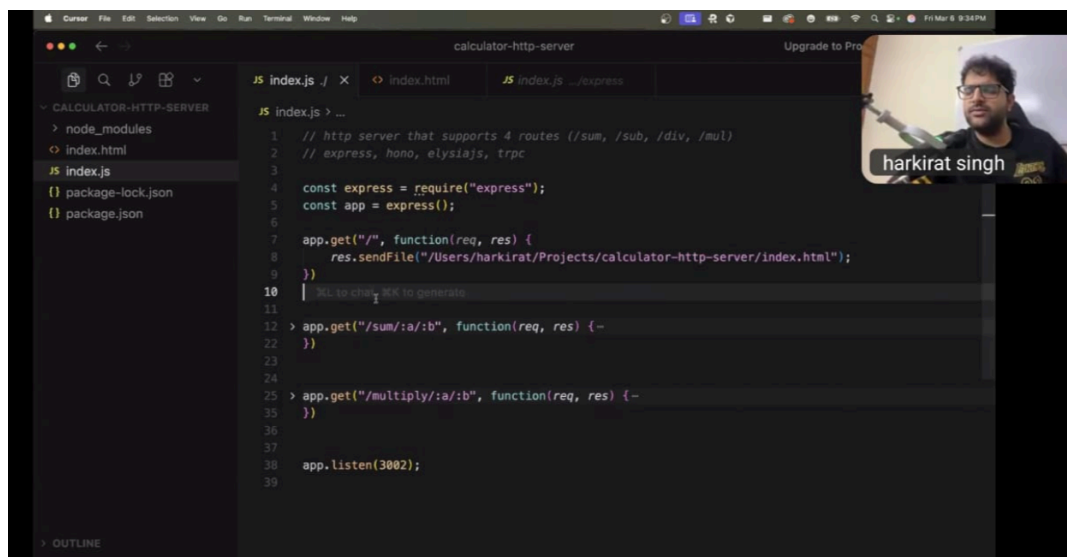
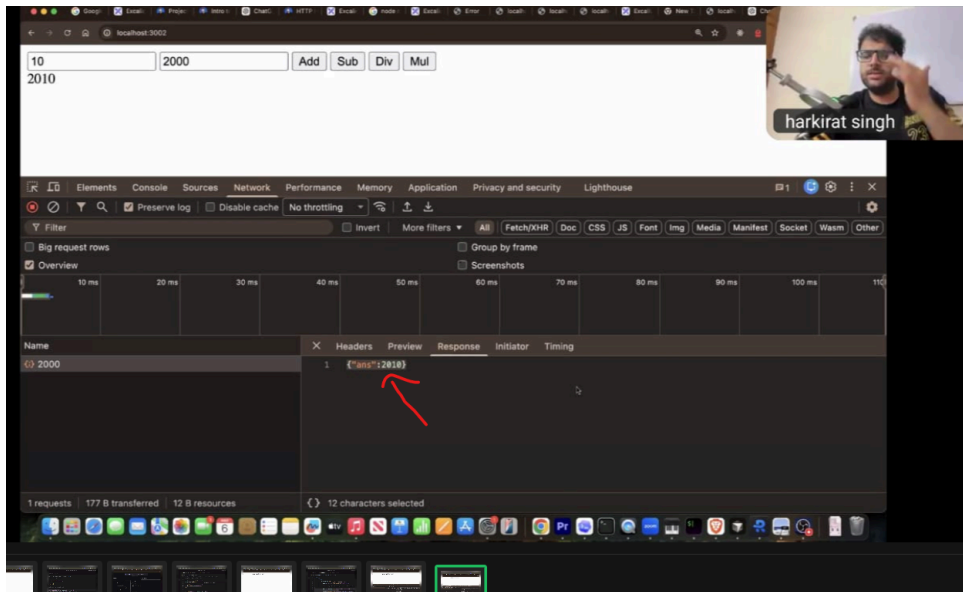
JS index.js ./ x index.html JS index.js ../express

CALCULATOR-HTTP-SERVER
> node_modules
< index.html
JS index.js
{} package-lock.json
{} package.json

JS index.js > app.get("/") callback
1 // http server that supports 4 routes (/sum, /sub, /div, /mul)
2 // express, hono, elysiajs, trpc
3
4 const express = require("express");
5 const app = express();
6
7 app.get("/", function(req, res) {
8   res.sendFile("/index.html");
9 })
10
11
12 app.get("/sum/:a/:b", function(req, res) {
13   const a = parseInt(req.params.a); // string 1
14   const b = parseInt(req.params.b); // string 2
15
16   const sum = a + b;
17
18   res.json({
19     ans: sum
20   })
21 })
22
23
24
25 > app.get("/multiply/:a/:b", function(req, res) {~
35 })
```

A screenshot of a code editor showing the implementation of an HTTP server using Express.js. The code defines routes for sum, sub, div, and mul. A video inset in the top right corner shows a person named harkirat singh.





```
calculator-http-server
Upgrade to Pro

JS index.js
index.html
JS index.js .../express

CALCULATOR-HTTP-SERVER
> node_modules
> index.html
JS index.js
() package-lock.json
() package.json

JS index.js > ...
1 // http server that supports 4 routes (/sum, /sub, /div, /mul)
2 // express, hono, elysiajs, trpc
3
4 const express = require("express");
5 const app = express();
6
7 app.use(express.json());
8
9 app.get("/", function(req, res) {
10   res.sendFile("/Users/harkirat/Projects/calculator-http-server/index.html");
11 })
12
13
14 app.post("/sum", function(req, res) {
15   const a = parseInt(req.body.a); // string 1
16   const b = parseInt(req.body.b); // string 2
17
18   const sum = a + b;
19
20   res.json({
21     ans: sum
22   });
23 })
24
25
26
```

```
calculator-http-server
Upgrade to Pro

JS index.js
index.html 2
JS index.js .../express

index.html > html > head > script > add > res > body
1 <html>
2   <head>
3     <script>
4       async function add() {
5         const a = document.getElementById("first").value;
6         const b = document.getElementById("second").value;
7
8         // document.getElementById("response").innerHTML = parseInt(a) + parseInt(b);
9         // axios
10        const res = await fetch("http://localhost:3002/sum/", {
11          method: "POST",
12          headers: {
13            "Content-Type": "application/json"
14          },
15          body: "{a: 1, b: 2}"
16        });
17        const json = await res.json()
18        document.getElementById("response").innerHTML = json.ans;
19      }
20    }
21  }
22 </script>
23 </head>
24 <body>
25   <input id="first" type="text" placeholder="first number" />
26   <input id="second" type="text" placeholder="second number" />

```

calculator-http-server

index.html

```
1 <html>
2 <head>
3 <script>
4   async function add() {
5     const a = document.getElementById("first").value;
6     const b = document.getElementById("second").value;
7
8     // document.getElementById("response").innerHTML = parseInt(a) + parseInt(b);
9
10    const res = await fetch("http://localhost:3002/sum/", {
11      method: "POST",
12      headers: {
13        "Content-Type": "application/json"
14      },
15      body: JSON.stringify({
16        a: a,
17        b: b
18      })
19    });
20    const json = await res.json();
21    document.getElementById("response").innerHTML = json.ans;
22  }
23 </script>
24 </head>
25 <body>
```

harkirat singh

localhost:3002

10 20 Add Sub Div Mul

30

harkirat singh

Elements Console Sources Network Performance Memory Application Privacy and security Lighthouse

Filter No throttling

Overview

10 ms 20 ms 30 ms 40 ms 50 ms 60 ms 70 ms 80 ms 90 ms 100 ms 110 ms

Name Headers Payload Preview Response Initiator Timing Cookies

sum/

General

Request URL http://localhost:3002/sum/

Request Method POST

Status Code 200 OK

Remote Address [::1]:3002

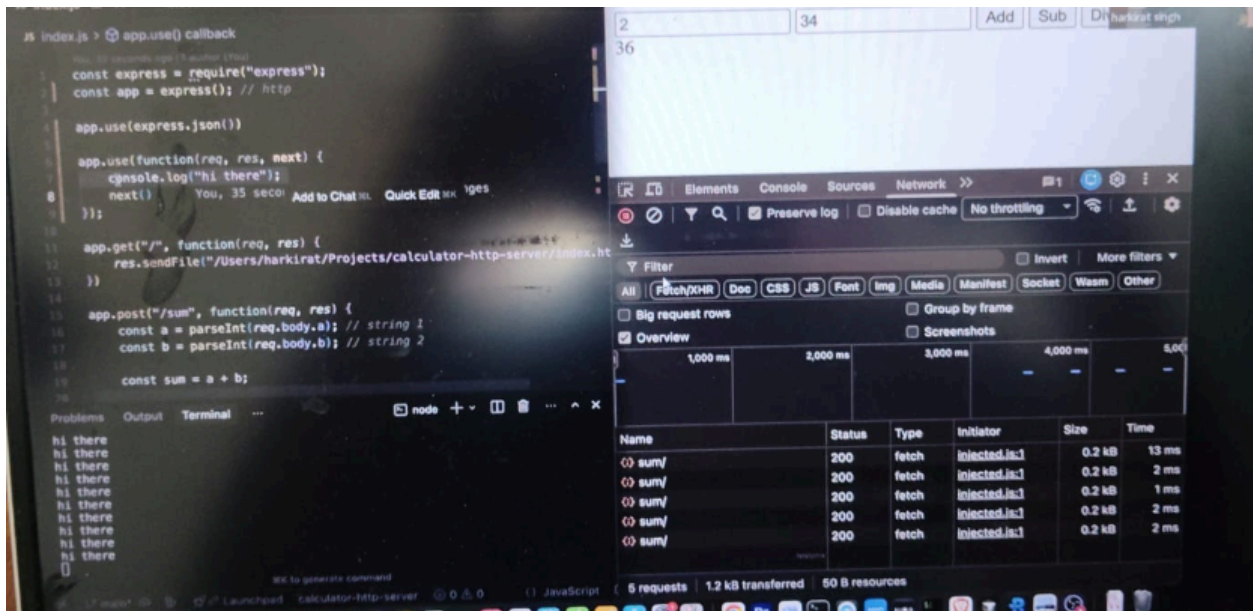
Referrer Policy strict-origin-when-cross-origin

Response headers (7)

Request Headers Raw

1 requests 244 B transferred 10 B resources

8. Middleware:



Middleware runs **before the request reaches the final route handler**. It sits between the **incoming request** and the **server response** and can **modify or process the request and response**.

Middleware can be used for tasks like:

- Parsing request data
- Authentication
- Logging
- Modifying request or response objects

For example, in **Express.js**, **`express.json()`** is a middleware that **parses JSON data from the request body**.

If we do **not use `express.json()`**, then Express will **not parse the JSON body**, and **`req.body`** will be **undefined or empty**.

Example:

Without middleware:

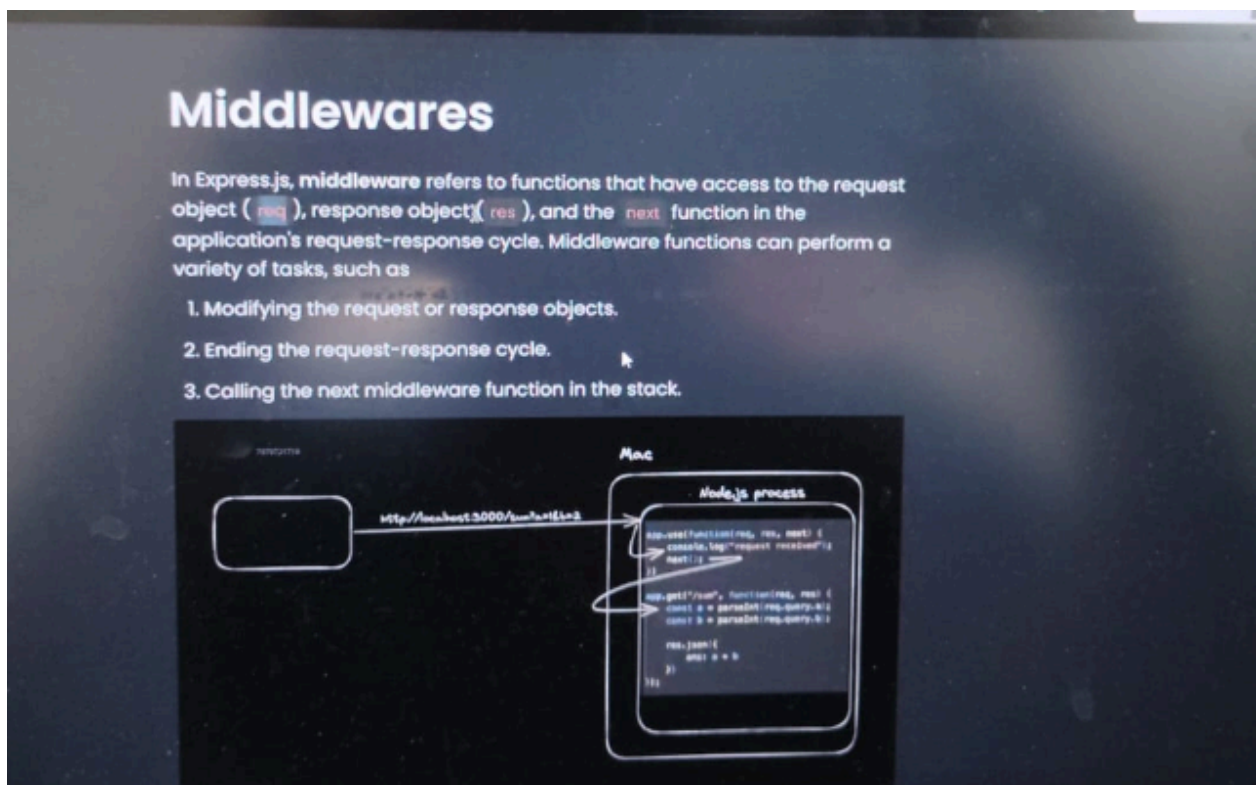
```
app.post("/user", (req, res) => {  
  console.log(req.body); // undefined  
});
```

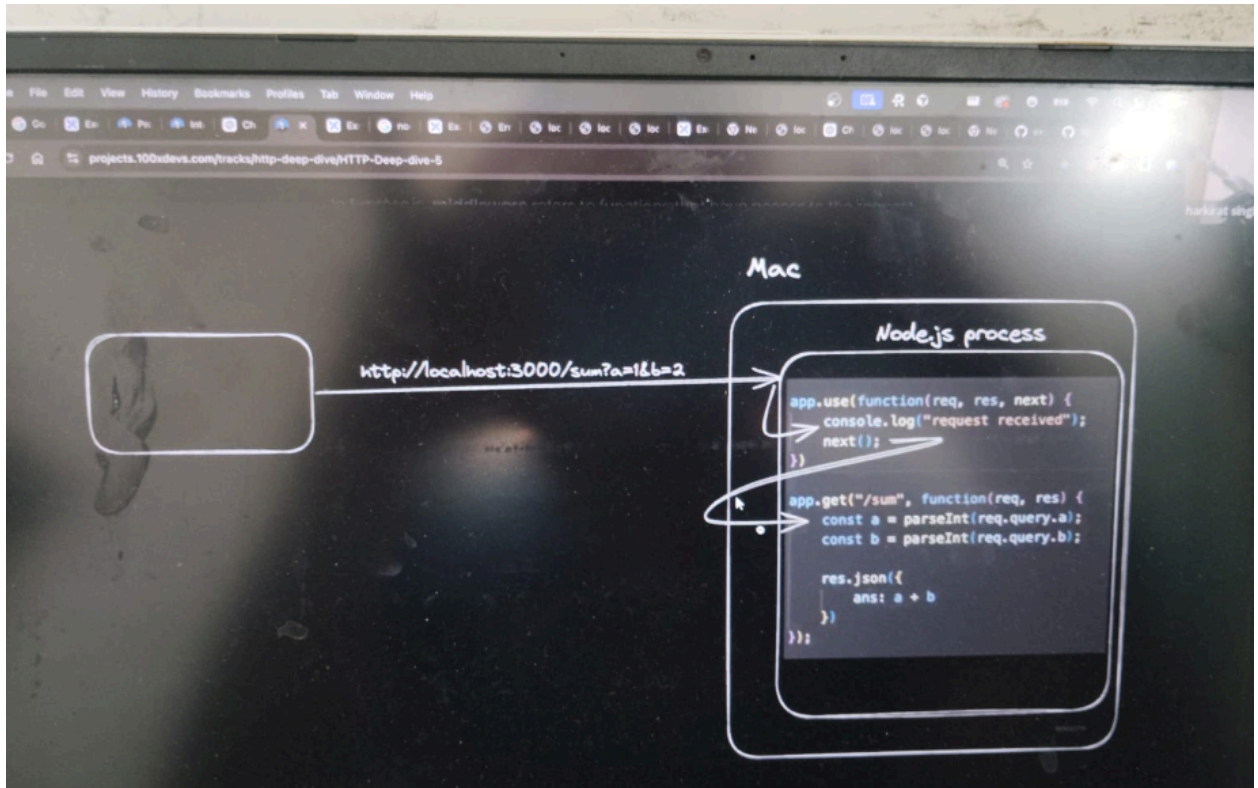
With middleware:

```
app.use(express.json());  
  
app.post("/user", (req, res) => {  
  console.log(req.body); // parsed JSON object  
});
```

Simple flow:

Client Request → Middleware → Route Handler → Response





Example 1:

```
const express = require("express");
const app = express();

app.use(express.json());
let requestCount = 0;

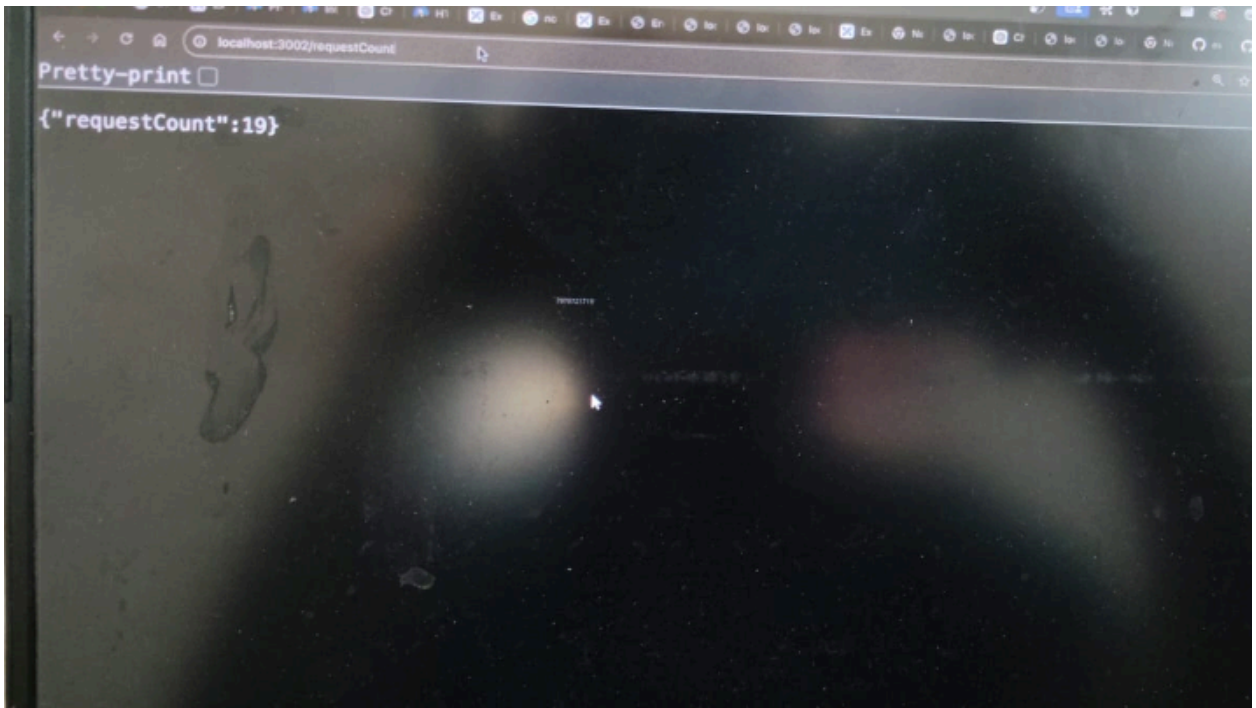
app.post("/multiply", function(req, res) {
  requestCount++;
  const a = parseInt(req.body.a); // string 1
  const b = parseInt(req.body.b); // string 2

  const ans = a * b;

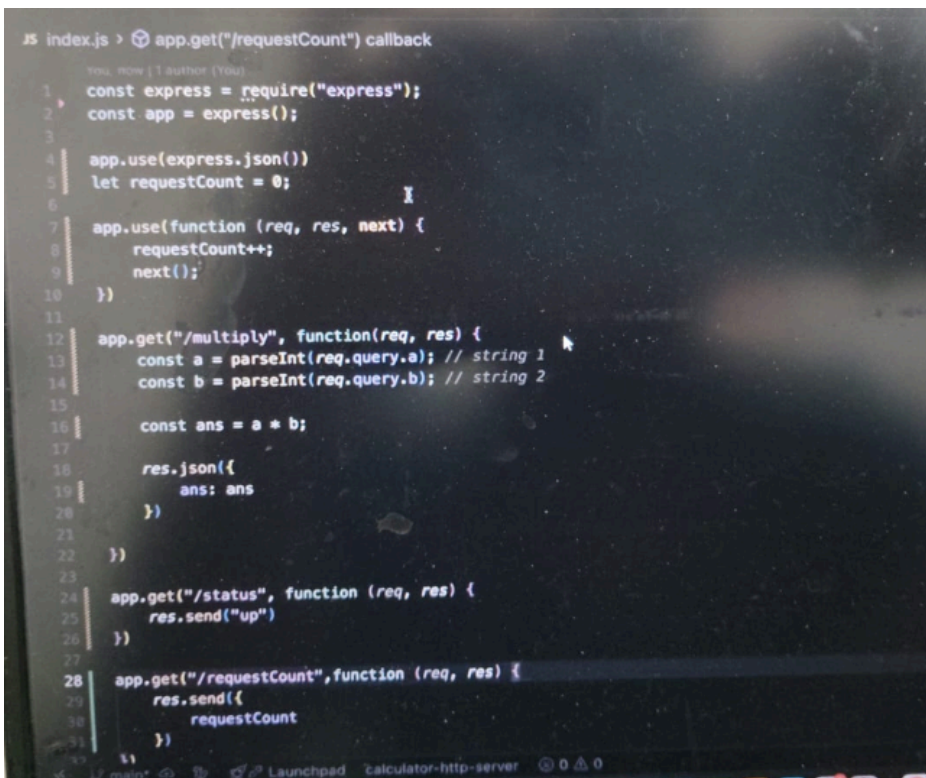
  res.json({
    ans: ans
  });
});

app.get("/status", function (req, res) {
  requestCount++;
  res.send("up")
});

app.get("/requestCount", function (req, res) {
  requestCount++;
  res.send({
    requestCount
  });
});
```

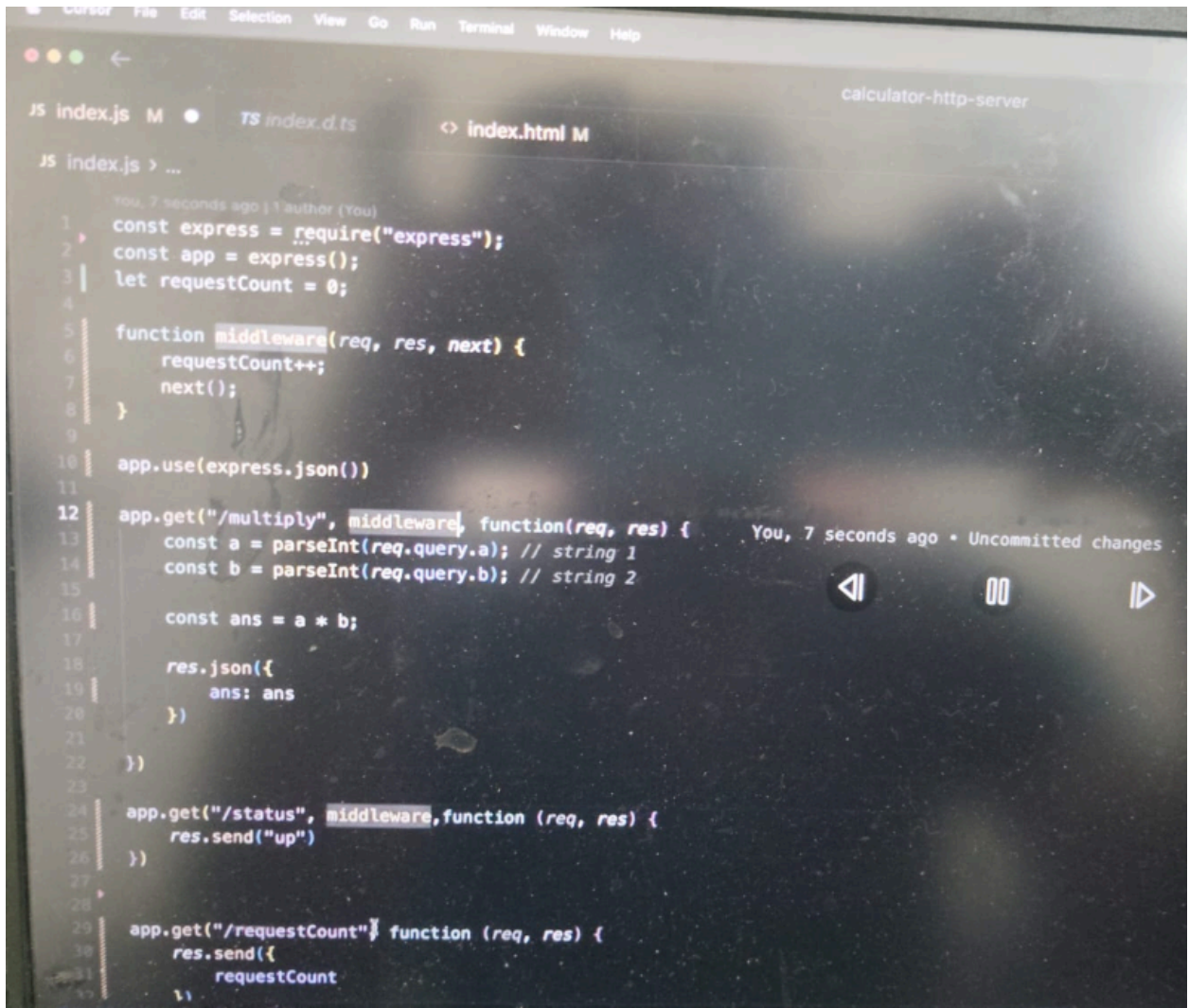



Above is not good way, Now below is good way



9. Route specific middlewares

<https://projects.100xdevs.com/tracks/http-deep-dive/HTTP-Deep-dive-6>



The screenshot shows a code editor with a dark theme. The file explorer on the left shows 'index.js' selected. The main editor area displays the following JavaScript code:

```
1 const express = require("express");
2 const app = express();
3 let requestCount = 0;
4
5 function middleware(req, res, next) {
6   requestCount++;
7   next();
8 }
9
10 app.use(express.json())
11
12 app.get("/multiply", middleware, function(req, res) {
13   const a = parseInt(req.query.a); // string 1
14   const b = parseInt(req.query.b); // string 2
15
16   const ans = a * b;
17
18   res.json({
19     ans: ans
20   })
21 })
22
23
24 app.get("/status", middleware, function (req, res) {
25   res.send("up")
26 })
27
28
29 app.get("/requestCount") function (req, res) {
30   res.send({
31     requestCount
```

The code implements an Express.js application. It starts by requiring 'express' and creating an 'app' instance. A 'requestCount' variable is initialized to 0. A 'middleware' function is defined to increment 'requestCount' and call 'next()'. This middleware is applied globally using 'app.use(express.json())'. Then, three routes are defined: 1. '/multiply' uses the 'middleware' and returns a JSON object with the product of 'a' and 'b' from the query. 2. '/status' uses the 'middleware' and sends 'up'. 3. '/requestCount' uses the 'middleware' and sends the 'requestCount' value.

The screenshot shows a code editor with a file named `index.js`. The code defines an Express application with a middleware function and two routes. The middleware function increments a `requestCount` variable and calls `next()` to pass control to the next function in the stack. The first route, `/multiply`, takes two query parameters, `a` and `b`, calculates their product, and returns it as JSON. The second route, `/status`, simply returns `"up"`. A third route, `/requestCount`, is partially visible at the bottom of the screen.

```
1 const express = require("express");
2 const app = express();
3 let requestCount = 0;
4
5 function middleware(req, res, next) {
6   requestCount++;
7   next();
8 }
9
10 app.use(express.json())
11
12 app.get("/multiply", middleware, function(req, res) {
13   const a = parseInt(req.query.a); // string 1
14   const b = parseInt(req.query.b); // string 2
15
16   const ans = a * b;
17
18   res.json({
19     ans: ans
20   })
21 })
22
23
24 app.get("/status", middleware, function (req, res) {
25   res.send("up")
26 })
27
28
29 app.get("/requestCount", function (req, res) {
30   res.send({
31     requestCount
32   })
33 })
```

10. When to use middleware

Calling the next middleware function in the stack.

```
app.use(function(req, res, next) {
  console.log("request received");
  next();
})

app.get("/sum", function(req, res) {
  const a = parseInt(req.query.a);
  const b = parseInt(req.query.b);

  res.json({
    ans: a + b
  })
});
```


This below we use generally during authentication

Ending the request/response cycle

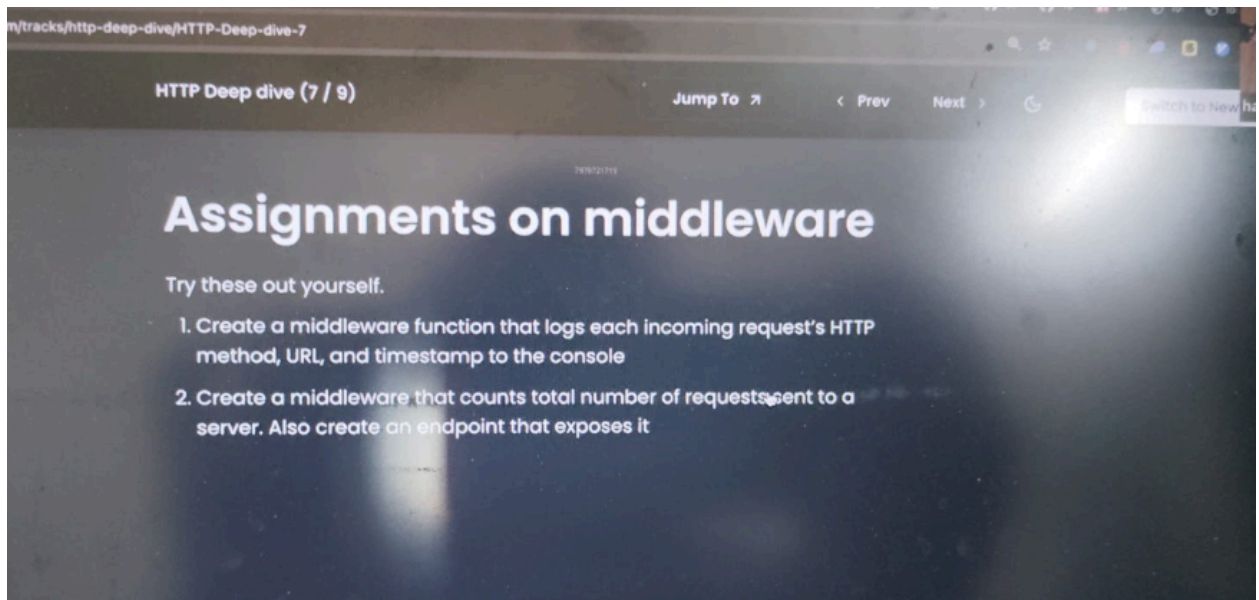
```
app.use(function(req, res, next) {  
  res.json({  
    message: "You are not allowed"  
  })  
})  
  
app.get("/sum", function(req, res) {  
  console.log(req.name);  
  const a = parseInt(req.query.a);  
  const b = parseInt(req.query.b);  
  
  res.json({  
    ans: a + b  
  })  
});
```

We can also modify the request (like express.json is doing)

Modifying the request

```
app.use(function(req, res, next) {  
  req.name = "harkirat"  
  next();  
})  
  
app.get("/sum", function(req, res) {  
  console.log(req.name);  
  const a = parseInt(req.query.a);  
  const b = parseInt(req.query.b);  
  
  res.json({  
    ans: a + b  
  })  
});
```

11. Assignment we have to do



The screenshot shows a web browser window with a dark theme. The address bar at the top displays the URL `mytracks/http-deep-dive/HTTP-Deep-dive-7`. The page header includes the title **HTTP Deep dive (7 / 9)**, a **Jump To** link, and navigation buttons for **< Prev** and **Next >**. A **Switch to New** button is visible on the right. The main content area has a large heading **Assignments on middleware** and a subtext **Try these out yourself.** Below this, there are two numbered tasks:

1. Create a middleware function that logs each incoming request's HTTP method, URL, and timestamp to the console
2. Create a middleware that counts total number of requests sent to a server. Also create an endpoint that exposes it